Contributing to BrainStorm

Thanks for your interest in improving BrainStorm! This document explains how to set up the project, the conventions we follow, and how to get a change merged.

- Code of conduct
- Security policy — please report vulnerabilities privately, **not** as a public issue

Development setup

```
git clone https://github.com/geff1778/Brainstorm-test.git
cd Brainstorm-test

python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install -r requirements.txt
pip install -r requirements-dev.txt
cp .env.example .env             # edit values; never commit .env
```

Run the app and the checks:

```
python run.py                    # http://localhost:5000
pytest                           # full suite (190 tests)
pytest --cov=app --cov-report=term-missing
ruff check app tests             # lint
ruff format app tests            # format
```

The test suite runs on the stdlib `threading` Socket.IO async mode against a throwaway `DATA_DIR`, so it needs no external services.

Branching & commits

- Branch off `main`. Use a descriptive name: `feat/…`, `fix/…`, `docs/…`, `refactor/…`.
- Keep commits focused. Write imperative subject lines, e.g. `fix: guard double question resolve`.
- Never commit `.env`, `data/`, databases or any credential — `.gitignore` covers the common cases.
- Do not push directly to `main`; open a pull request instead.

Architecture rules

The project uses a layered architecture. Please respect the dependency direction:

```
config / security / validation / errors / logging_setup   (cross-cutting)
        ▲
domain  (pure game logic — no I/O, no network, no DB)
        ▲
services (application logic; may use domain + db)
        ▲
realtime / http_api (thin adapters over services)
```

- `domain` must stay pure and import nothing from `services`, `realtime` or `http_api`.
- Put side effects (DB, network, Socket.IO) in `services`, `realtime` or `http_api`.
- Prefer small functions and dependency injection over globals.

Code style

- Python: PEP 8 via `ruff` (config in `pyproject.toml`, line length 110). Add type hints and concise doc-strings to public functions, classes and modules.
- JavaScript: plain ES modules in `static/js`, 2-space indent, `node --check` must pass.
- Keep comments meaningful — explain why, not what.

Tests

- Every bug fix and new feature should come with a test.
- Unit-test real code paths; avoid mocks unless unavoidable (and justify them).
- `tests/conftest.py` provides fixtures (`app`, `client`, `socket_client`, `socket_factory`, `admin_client`, `tester_client`) — reuse them.

Pull requests

1. Make sure `ruff check app tests`, `pytest` and `node --check static/js/game.js` all pass.
2. Fill in the PR template; describe the problem, the approach and how you verified it.
3. Link any related issue.
4. A maintainer will review. Please respond to review comments; keep the branch up to date.

By contributing you agree that your contributions are licensed under the project's MIT License.